

RAPPORT DE TP

Communication, APIs et fiabilité

Contrat d'API, politiques de résilience et sécurité du SGDD

Module : Applications Réparties et Cybersécurité — Séance 6
1ère année Cybersécurité — 2025/2026

1. Introduction

Cette séance prolonge directement la séance 5, consacrée à l'architecture du Système de Gestion Documentaire Distribué (SGDD), composé de trois services (Auth Service, Document Service, Search Service) derrière un API Gateway. L'objectif de ce TP était d'ajouter trois couches supplémentaires à cette architecture : le contrat d'API (endpoints, méthodes, payloads, codes d'erreur, idempotence), les politiques de fiabilité côté client (timeouts, retries, backoff, idempotency keys) avec l'analyse de trois scénarios de panne, et une première matrice de sécurité (surfaces d'exposition, menaces, contrôles).

2. TP6.1 — Contrat d'API

Le tableau suivant définit le contrat d'API complet pour les trois services du SGDD, avec versioning par chemin d'URI (/api/v1/...).

Endpoint	Méthode	Entrée (JSON)	Sortie (JSON)	Erreurs	Idem p.	Sécurité
/api/v1/auth/login	POST	{username: str, password: str}	{token: str, expires_at: str}	400, 401, 429	Non	Rate limiting strict, TLS, message d'erreur générique
/api/v1/auth/verify	GET	Header Authorization: Bearer <token>	{valid: bool, user_id: str, roles: [str]}	401	Oui	Utilisé en inter-service par Documents et Search
/api/v1/auth/logout	POST	Header Authorization: Bearer <token>	204 No Content	401	Oui	Invalide le token côté Auth Service
/api/v1/documents	POST	{title: str (1-200), content: str (max 50000), tags: [str] (max 10)}	{id: UUID, title: str, created_at: str}	400, 401, 403, 413, 429	Non (Idempotency-Key)	AuthN + AuthZ rôle éditeur, validation stricte, taille max
/api/v1/documents	GET	Query: page, per_page (max 100), sort, order, tag	{data: [...], total: int, page: int, per_page: int}	400, 401	Oui	AuthN, pagination forcée côté serveur
/api/v1/documents/{id}	GET	Path: id (UUID)	{id, title, content, tags, created_at, updated_at}	401, 403, 404	Oui	AuthN + AuthZ propriétaire ou rôle lecteur
/api/v1/documents/{id}	PUT	{title: str, content: str, tags: [str]}	{id, title, updated_at}	400, 401, 403, 404, 409	Oui	AuthN + AuthZ propriétaire ou éditeur
/api/v1/documents/{id}	DELETE	Path: id (UUID)	204 No Content	401, 403, 404	Oui	AuthN + AuthZ propriétaire ou admin, audit log
/api/v1/search	GET	Query: q, page, per_page (max 100), tag, date_from,	{results: [...], total: int, page: int}	400, 401, 429	Oui	AuthN, rate limiting (requêtes coûteuses),

		date_to			résultats filtrés par AuthZ
--	--	---------	--	--	--------------------------------

2.1 Remarques sur l'idempotence

Seuls deux endpoints ne sont pas idempotents par nature : /auth/login (chaque appel génère un nouveau token, ce qui est le comportement attendu) et POST /documents (chaque appel crée un nouveau document). Pour ce dernier, un retry suite à un timeout réseau pourrait créer un document en double : le client devra donc générer un UUID côté client et l'envoyer dans un header Idempotency-Key. Le Document Service stocke cette clé associée au document créé ; si la même clé revient (retry), il retourne directement le document déjà créé (avec le même code 201 ou 200) au lieu d'en créer un second.

2.2 Identifiants de ressources

Conformément à la bonne pratique vue en cours, tous les identifiants de documents sont des UUID v4 (aléatoires, non séquentiels), afin d'éviter qu'un attaquant ne puisse énumérer les documents existants en testant des identifiants consécutifs (1, 2, 3...).

3. TP6.2 — Politiques de fiabilité côté client

3.1 Politiques par service

Service / opération	Timeout	Max retries	Base delay / jitter	Codes retryables	Idempo tency key ?
Auth — login	5 s	1	1 s / pas de jitter (peu d'appels concurrents)	502, 503, 504	Non
Auth — verify	3 s	2	0,5 s / full jitter	502, 503, 504	Non (lecture, GET idempot ent)
Documents — GET (liste / par id)	10 s	2	1 s / full jitter	500, 502, 503, 504	Non (lecture)
Documents — POST (création)	15 s	0 (pas de retry auto)	—	—	Oui (UUID généré côté client)
Documents — PUT / DELETE	10 s	1	1 s / full jitter	503, 504	Oui (idempo tents par nature)
Search — query	8 s	2	1 s / full jitter	502, 503	Non (lecture)

Le timeout de Documents POST (15 s) est plus élevé que celui des lectures (10 s) car la création d'un document peut impliquer une écriture sur le stockage objet et les métadonnées,

ce qui prend naturellement plus de temps. Aucun retry automatique n'est appliqué sur ce endpoint sans Idempotency-Key, conformément au piège classique vu en cours : un timeout sur un POST ne garantit pas que le serveur n'a pas traité la requête. Le verify d'Auth a un timeout court (3 s) et 2 retries car il est appelé en inter-service par Documents et Search à chaque requête : il doit être rapide et résilient pour ne pas devenir lui-même un goulot d'étranglement.

3.2 Analyse des trois scénarios

Scénario	Risque principal	Politique appliquée	Justification
S1 — Latence élevée (Document Service répond en 8-15 s au lieu de 200 ms)	Le thread/connexion du client reste bloqué longtemps ; si de nombreux utilisateurs sont impactés simultanément, les ressources du Gateway (sockets, threads) s'épuisent et l'effet se propage en cascade vers les autres appels.	Timeout fixé à 10 s pour les lectures Documents (S1 dépasse ce seuil et déclenche donc un échec contrôlé plutôt qu'une attente indéfinie). Pas de retry immédiat agressif : un seul retry avec backoff. Si le circuit breaker associé au Document Service détecte un taux d'échec élevé, il s'ouvre et bascule en fallback (réponse en cache ou message 'service temporairement indisponible').	Un timeout strict évite la famine de ressources côté Gateway. Le circuit breaker empêche que la lenteur d'un seul service ne contamine Auth et Search via l'épuisement du pool de connexions partagé. Le fallback en cache permet à l'utilisateur de continuer à consulter une liste de documents légèrement obsolète plutôt que de voir une erreur brute.
S2 — Serveur intermittent (Search Service retourne 503 une requête sur trois pendant un rolling update)	Sans retry, un utilisateur sur trois voit une erreur de recherche alors que le service fonctionne en réalité dans 2 cas sur 3. Avec un retry naïf sans backoff, le pic de requêtes pendant le déploiement peut aggraver la situation (retry storm).	503 fait partie des codes retryables pour Search (cf. tableau 3.1) : le client retente jusqu'à 2 fois avec un backoff exponentiel (1 s, 2 s) et full jitter. Comme 2 requêtes sur 3 réussissent, un seul retry suffit statistiquement dans la grande majorité des cas.	503 est par définition une erreur transitoire (service temporairement indisponible), c'est le cas typique où le retry est utile et recommandé. Le jitter évite que tous les clients impactés au même instant par le rolling update ne retentent simultanément, ce qui ajouterait de la charge précisément pendant le déploiement.
S3 — Duplication de requêtes (double clic sur 'Créer un document', deux POST /documents identiques partent avant la première réponse)	Deux documents identiques sont créés en base, ce qui pollue les résultats de recherche, peut induire l'utilisateur en erreur (lequel est le 'vrai' ?) et complique l'audit.	Le client génère un UUID unique (Idempotency-Key) au moment du clic sur 'Créer', et l'envoie dans le header de chaque tentative liée à cette action (y compris si le second clic relance un nouvel appel). Le Document Service stocke cette clé associée au document créé ; si la même clé est reçue à nouveau, il renvoie le document déjà créé sans en créer un second. Côté UX, le bouton 'Créer' est désactivé dès le premier clic.	C'est exactement le cas d'usage de l'Idempotency-Key décrit en cours pour POST /documents : sans elle, aucune politique de retry ou de timeout ne peut empêcher la duplication, car le problème vient du client (double appel), pas du réseau. La désactivation du bouton est une mesure complémentaire côté UX, mais l'Idempotency-Key reste la seule garantie côté serveur, valable même en cas de double clic très rapproché ou de re-soumission par un script.

3.3 Situations de fallback identifiées

- Documents — GET (liste) : en cas d'indisponibilité prolongée du Document Service, retourner la dernière liste en cache (avec un indicateur 'données potentiellement obsolètes') plutôt qu'une erreur brute.
- Search — query : en cas de 503 répétés au-delà des retries, retourner un message clair 'recherche temporairement indisponible, réessayez dans quelques instants' plutôt qu'une page d'erreur générique, le reste de l'application (consultation, upload) restant fonctionnel.
- Auth — verify : si Auth Service est totalement indisponible (et non simplement lent), aucun fallback n'est acceptable pour les opérations sensibles (écriture) ; en revanche, pour les opérations de lecture déjà en cache côté Gateway (résultat de verify mis en cache quelques minutes), on peut tolérer un token validé récemment, dans une fenêtre de temps courte, pour ne pas bloquer totalement la lecture.

4. TP6.3 — Sécurité API : modèle minimal

4.1 Mécanisme de token

- Génération : à la connexion réussie (/auth/login), l'Auth Service génère un token signé (JWT), incluant l'identifiant utilisateur, ses rôles et une date d'expiration (exp), signé avec une clé secrète connue uniquement de l'Auth Service (et, pour la vérification locale, partagée via clé publique si signature asymétrique).
- Transmission : le token est transmis dans le header Authorization: Bearer <token> sur chaque requête, exclusivement via TLS (HTTPS).
- Vérification : pour les opérations sensibles, chaque service (Documents, Search) appelle /auth/verify (vérification centralisée) ; pour réduire la latence sur les opérations fréquentes, les services peuvent également vérifier localement la signature du JWT avec la clé publique d'Auth, sans appel réseau supplémentaire, à condition de vérifier systématiquement l'expiration (exp) et l'audience (aud).
- Invalidation : un token expire automatiquement après une durée courte (par exemple 1 heure). Le logout (/auth/logout) ajoute le token à une liste de révocation consultée par /auth/verify jusqu'à son expiration naturelle, ce qui permet une invalidation immédiate même si le token n'est pas encore expiré.

4.2 Placement des contrôles de sécurité

- Validation d'entrée : appliquée aux deux niveaux. L'API Gateway effectue une première validation de surface (format JSON, taille du payload, présence des headers requis), mais chaque service (Documents, Search) revalide intégralement ses propres entrées (types, longueurs, valeurs autorisées) sans faire confiance au Gateway seul, conformément au principe Zero Trust.
- Rate limiting : appliqué sur /auth/login de manière stricte (5 tentatives par minute par IP, avec verrouillage progressif), sur /api/v1/search de manière modérée (car les requêtes de recherche sont coûteuses en CPU pour Elasticsearch), et plus largement sur l'ensemble des endpoints au niveau du Gateway (quota global par utilisateur/IP) pour limiter le DDoS applicatif.
- Audit logs : événements journalisés — connexions réussies et échouées (login), créations et suppressions de documents (avec utilisateur, document, timestamp), accès

aux endpoints d'administration, et tout refus d'autorisation (403). Les tokens et mots de passe ne sont jamais inclus dans les logs.

4.3 Matrice Surface → Menace → Contrôle → Priorité

Surface	Menace principale	Contrôle proposé	Priorité
API publique — /auth/login	Brute force sur les credentials	Rate limiting (5 tentatives/min/IP), verrouillage progressif, message d'erreur générique (pas de distinction 'utilisateur inexistant' vs 'mot de passe incorrect'), audit log	Critique
API publique — /documents (POST)	Injection JSON, upload de contenu malveillant ou trop volumineux	Validation stricte des champs (title, content, tags), taille maximale du payload (413 si dépassée), vérification du Content-Type	Critique
API publique — /search	DDoS applicatif via requêtes coûteuses répétées	Rate limiting dédié, pagination obligatoire (per_page max 100 côté serveur), timeout serveur sur Elasticsearch	Élevée
API publique — tous endpoints	MITM, interception de tokens en transit	TLS obligatoire sur tous les endpoints (y compris internes), en-tête HSTS	Critique
Inter-services — Auth ↔ Documents ↔ Search	Mouvement latéral, usurpation d'identité d'un service	mTLS entre tous les services ou tokens de service dédiés, vérification de l'audience du JWT à chaque appel	Élevée
Inter-services — tous	Replay d'une requête interne interceptée	Nonces, timestamps avec fenêtre de validité courte, idempotency keys déjà en place pour les écritures	Moyenne
Admin — gestion des utilisateurs et des rôles	Accès non autorisé, élévation de privilèges (ex : un utilisateur modifie son propre rôle en 'admin')	AuthZ stricte limitée au rôle admin, vérification du rôle côté serveur à chaque opération sensible (jamais déduit du payload client seul), MFA pour les comptes admin, audit log renforcé	Critique
Admin — endpoints de statistiques et debug	Fuite d'informations sensibles sur l'architecture interne	Non exposés publiquement (accessibles uniquement depuis le réseau interne), protégés par authentification + autorisation admin	Élevée

Cette matrice est cohérente avec les annotations de sécurité du contrat d'API (section 2) : par exemple, le rate limiting strict sur /auth/login et la validation stricte sur POST /documents apparaissent dans les deux livrables, et l'AuthN + AuthZ propriétaire/éditeur du contrat d'API se traduit ici par le contrôle AuthZ stricte côté admin pour les opérations les plus sensibles.

5. Lab sécurité — STRIDE et incident 'token leaked'

5.1 Application de STRIDE au SGDD

Catégorie STRIDE	Exemple concret sur le SGDD
Spoofing	Un attaquant vole le token JWT d'un utilisateur (par exemple via un poste compromis) et l'utilise pour accéder à ses documents privés.
Tampering	Sans TLS, un attaquant en position MITM pourrait modifier le contenu d'un

	document pendant son transfert vers le Document Service.
Repudiation	Un utilisateur supprime un document sensible et nie l'avoir fait, faute d'audit log horodaté associant l'action à son identifiant.
Information Disclosure	Une erreur 500 en production renvoie une stack trace détaillée révélant la structure interne du Document Service (chemins de fichiers, version de bibliothèques).
Denial of Service	Un script envoie des milliers de requêtes GET /search?q=... sans pagination, saturant Elasticsearch et rendant la recherche indisponible pour tous.
Elevation of Privilege	Un utilisateur modifie le payload de sa requête pour tenter de s'attribuer le rôle 'admin', en espérant qu'un service ne revalide pas ce champ côté serveur.

5.2 Incident 'token leaked' : étapes de réponse

Scénario appliqué au SGDD : un fichier .env contenant le token de service utilisé par le Document Service pour appeler Auth Service en inter-service a été accidentellement commité dans un dépôt Git public, et découvert 3 heures plus tard.

1. Containment immédiat : révoquer sans délai le token compromis côté Auth Service, le rendant invalide pour toute requête future, même si le dépôt n'est pas encore nettoyé — un dépôt public peut être scanné par des bots en quelques secondes.
2. Rotation : générer un nouveau token de service et le déployer sur le Document Service, afin de rétablir la communication inter-services Documents → Auth avec des credentials sains.
3. Audit : analyser les logs d'accès des 3 dernières heures pour identifier toutes les requêtes ayant utilisé l'ancien token (quelles IPs, quelles actions sur quels documents), afin de déterminer si une exploitation a eu lieu.
4. Évaluation d'impact : si des accès suspects sont détectés sur des documents, identifier précisément lesquels et les utilisateurs concernés ; en cas de données personnelles affectées, notification dans le délai RGPD de 72 heures.
5. Nettoyage : supprimer le fichier .env du dépôt et réécrire l'historique Git (le secret reste sinon accessible dans les anciens commits) ; ajouter .env au .gitignore.
6. Prévention : mettre en place un outil de détection de secrets en CI/CD (pre-commit hooks ou scan automatisé) pour empêcher qu'un futur commit contenant un token ne soit accepté.
7. Post-mortem : rédiger un rapport d'incident factuel (timeline, cause racine, impact constaté, actions correctives), dans un esprit d'amélioration continue plutôt que de recherche de responsabilité individuelle.

6. Synthèse

Les trois livrables produits pour ce TP sont cohérents entre eux : le contrat d'API (section 2) identifie pour chaque endpoint son idempotence et ses besoins de sécurité ; les politiques de fiabilité (section 3) traduisent ces besoins en valeurs concrètes de timeout, retry et idempotency key, validées sur trois scénarios réalistes ; et la matrice de sécurité (section 4) couvre les trois surfaces d'exposition (API publique, inter-services, administration) avec des contrôles directement liés aux annotations du contrat d'API.

Les points clés à retenir de cette séance, appliqués au SGDD, sont les suivants : aucun appel réseau ne doit être sans timeout ; les retries ne sont utiles que pour les erreurs transitoires (5xx, 429) et jamais pour les 4xx (sauf 429) ; les opérations non idempotentes comme POST

/documents nécessitent une Idempotency-Key pour être sûres face aux retries et aux doubles soumissions ; AuthN et AuthZ sont deux vérifications distinctes et successives, appliquées à chaque service et non au seul Gateway (Zero Trust) ; et l'observabilité (request ID, logs structurés) est indispensable pour diagnostiquer les scénarios comme S1 et S2 en production.

Module Applications Réparties et Cybersécurité — Séance 6 — 1ère année Cybersécurité — 2025/2026